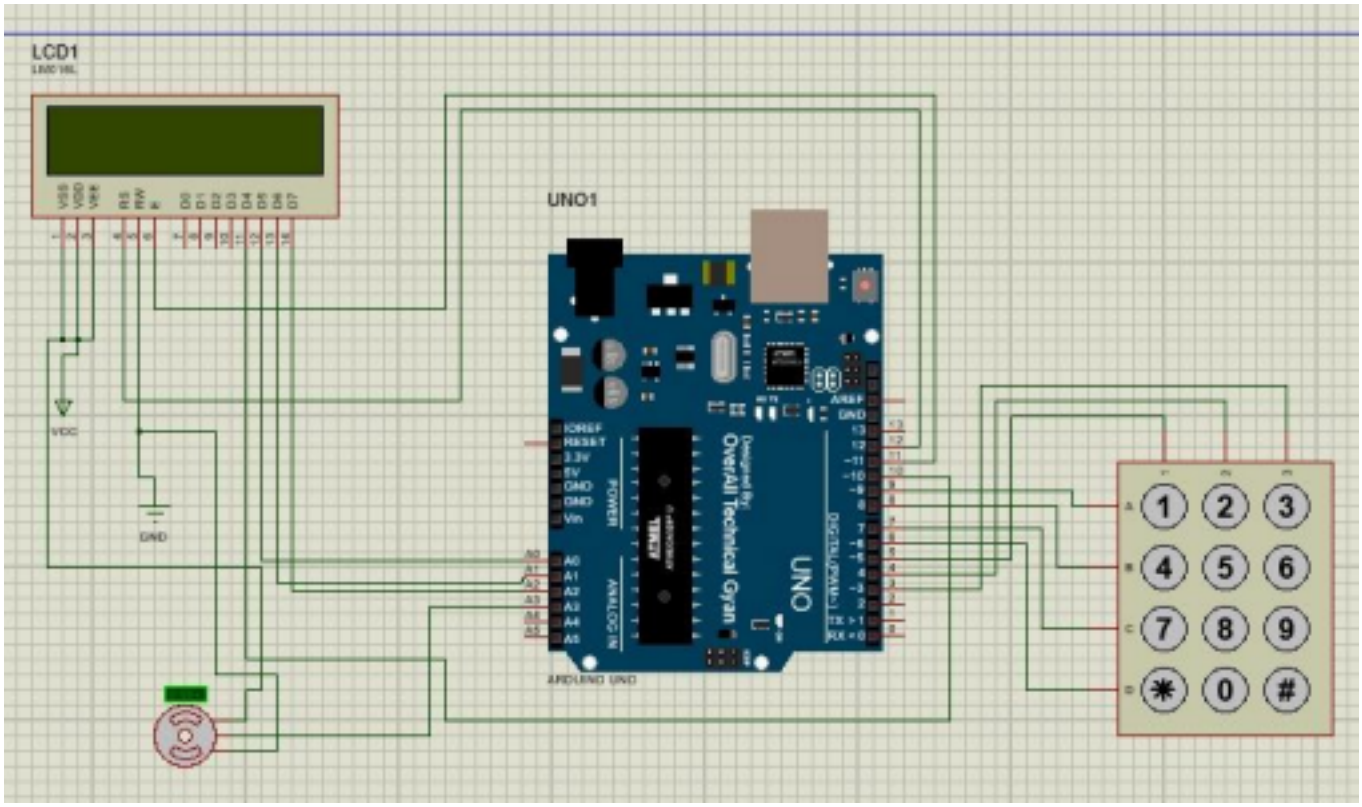
PASSWORD-BASED SMART DOOR LOCK SYSTEM

Complete Practical Guide

Arduino UNO • 4×3 Keypad • 16×2 LCD • Servo Motor
Circuit wiring • Full code • Explanation • Proteus simulation • Testing



Password-Based Smart Door Lock System - Lab Guide

1. Project Overview

This project is a password-based electronic door-lock demonstration. The user enters a password using a 4×3 matrix keypad. Arduino UNO compares the entered password with a password stored in the program. If the password is correct, a servo motor moves to the unlock position for five seconds and then returns to the locked position. If the password is incorrect, the LCD displays an access-denied message.

1.1 Main features

• Welcome message and password prompt on a 16×2 LCD.
• Password entry through a 4×3 matrix keypad.
• The LCD displays asterisks instead of the actual password digits.
• Press # to submit the password; press * to clear the input.

• Correct password: unlock servo for five seconds, then lock again.
• Incorrect password: display Wrong Password / Access Denied.

1.2 Components

Component	Qty.	Purpose
Arduino UNO	1	Main controller.
4×3 matrix keypad	1	User password input.
16×2 character LCD (e.g., LM016L)	1	Displays prompts and status.
Positional servo motor	1	Moves the lock mechanism.
10 kΩ potentiometer	1	LCD contrast adjustment.
Resistor for backlight (if required)	1	Limits LCD backlight current.
Wires and power source	As needed	Circuit connections.

Scope note: This final version does NOT use an ultrasonic sensor. It does not measure distance or automatically detect a person. The user starts authentication by entering a password.

2. Working Principle

1. Power is applied. Arduino initializes the LCD and servo and commands the servo to the locked position (0°).
2. The LCD displays a welcome message for about two seconds.
3. The LCD displays Enter Password: and Pass:.
4. The user enters digits on the keypad. Each digit appears as an asterisk (*).
5. The user presses # to submit the entered password.
6. Arduino compares the entered string with the stored password, which is 1234.
7. If correct, the LCD displays Access Granted / Door Unlocked and the servo moves to 90°.
8. After five seconds, the servo returns to 0° and the LCD displays Door Locked.
9. If incorrect, the LCD displays Wrong Password! / Access Denied and the servo stays locked.
10. The input is cleared and the password prompt appears again.

3. Pin Connections

Follow this pin mapping exactly; it matches the program in Section 5.

Module	Pin	Arduino UNO	Description
Keypad	R1	D9	Row 1
Keypad	R2	D8	Row 2
Keypad	R3	D7	Row 3
Keypad	R4	D6	Row 4
Keypad	C1	D5	Column 1
Keypad	C2	D4	Column 2
Keypad	C3	D3	Column 3
LCD	RS	D12	Register Select
LCD	EN	D11	Enable

LCD	D4	D10	LCD data bit 4
LCD	D5	A0	LCD data bit 5
LCD	D6	A1	LCD data bit 6

LCD	D7	A2	LCD data bit 7
Servo	Signal	A3	Servo control signal

3.1 LCD power and support pins

LCD pin	Connection	Purpose
VSS	GND	Ground.
VDD	+5 V	Power.
V0 / VEE	Potentiometer middle pin	Contrast; connect the other pot ends to +5 V and GND.
RW	GND	Write mode.
A / LED+	+5 V through resistor if required	Backlight anode.
K / LED−	GND	Backlight cathode.

The exact backlight labels and built-in resistor arrangement can vary between LCD models. Follow the labels on your Proteus component.

3.2 Servo connections

Servo wire	Connection
Signal (often orange/yellow/white)	Arduino A3
Positive (often red)	+5 V suitable for the servo
Ground (often brown/black)	GND; connect servo ground and Arduino GND together.

Hardware power note: A physical servo may draw more current than the Arduino 5 V pin can safely supply. Use a suitable regulated 5 V servo supply if needed, and connect its ground to Arduino GND. Check the actual servo datasheet and Proteus model pin labels.

4. Understanding the Code

4.1 Libraries

```
#include <Keypad.h>
#include <LiquidCrystal.h>
#include <Servo.h>
```

Keypad.h scans the matrix keypad. LiquidCrystal.h controls the character LCD in 4-bit mode. Servo.h generates the control pulses used to position the servo.

4.2 Keypad layout and pin arrays

```
const byte ROWS = 4;
const byte COLS = 3;

char keys[ROWS][COLS] = {
 {'1', '2', '3'},
 {'4', '5', '6'},
 {'7', '8', '9'},
 {'*', '0', '#'}
};

byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3};
```

ROWS and COLS define the keypad size. The keys array describes the character at each row/column intersection. rowPins and colPins tell the library which Arduino pins connect to the keypad lines.

4.3 LCD configuration

```
LiquidCrystal lcd(12, 11, 10, A0, A1, A2);
```

The arguments are RS, Enable, and LCD data pins D4, D5, D6, and D7, in that order. Arduino analog pins A0, A1, and A2 can also be used as digital I/O pins.

4.4 Servo and password variables

```
Servo lockServo;
const int servoPin = A3;
```



```
const int LOCK_POSITION = 0;
const int UNLOCK_POSITION = 90;

String password = "1234";
String input = "";
```

lockServo is the servo object. A3 is its signal pin. The program uses 0° as locked and 90° as unlocked; adjust these angles to suit the mechanical lock. password stores the correct password, while input stores the keys entered by the user.

5. Complete Arduino Code

Create a new Arduino sketch and paste the complete code below. It matches the pin table above and contains no ultrasonic sensor.

```
#include <Keypad.h>
#include <LiquidCrystal.h>
#include <Servo.h>

// 1. KEYPAD CONFIGURATION
const byte ROWS = 4;
const byte COLS = 3;

char keys[ROWS][COLS] = {
 {'1', '2', '3'},
 {'4', '5', '6'},
 {'7', '8', '9'},
 {'*', '0', '#'}
};

byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3};

Keypad keypad = Keypad(
 makeKeymap(keys),
 rowPins,
 colPins,
 ROWS,
 COLS
);

// 2. LCD CONFIGURATION
LiquidCrystal lcd(12, 11, 10, A0, A1, A2);

// 3. SERVO CONFIGURATION
Servo lockServo;
const int servoPin = A3;

const int LOCK_POSITION = 0;
const int UNLOCK_POSITION = 90;

// 4. PASSWORD CONFIGURATION
```

```
String password = "1234";
String input = "";

// 5. DISPLAY PASSWORD PROMPT
void passwordScreen() {
 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Enter Password:");

 lcd.setCursor(0, 1);
 lcd.print("Pass: ");
}

// 6. DISPLAY WELCOME MESSAGE
void welcomeScreen() {
 lcd.clear();

 lcd.setCursor(1, 0);
 lcd.print("Welcome To");

 lcd.setCursor(0, 1);
 lcd.print("Smart Door Lock");

 delay(2000);

 passwordScreen();
}

// 7. INITIALIZATION
void setup() {
 lcd.begin(16, 2);

 lockServo.attach(servoPin);
  lockServo.write(LOCK_POSITION);

 welcomeScreen();
}

// 8. MAIN PROGRAM
void loop() {
 char key = keypad.getKey();

 if (key) {

 // SUBMIT PASSWORD
 if (key == '#') {

 if (input == password) {
 // Correct password
 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Access Granted");

 lcd.setCursor(0, 1);
 lcd.print("Door Unlocked");

 lockServo.write(UNLOCK_POSITION);
delay(5000);

 lockServo.write(LOCK_POSITION);

 lcd.clear();
 lcd.setCursor(0, 0);
 lcd.print("Door Locked");

 delay(1500);
 }
 else {
 // Incorrect password
 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("Wrong Password!");

 lcd.setCursor(0, 1);
```

```
lcd.print("Access Denied");

delay(2000);
}

input = "";
passwordScreen();
}

// CLEAR INPUT
else if (key == '*') {
```



```
input = "";

lcd.setCursor(0, 1);
lcd.print(" ");

lcd.setCursor(0, 1);
lcd.print("Pass: ");
}

// ADD A PASSWORD DIGIT
else {
if (input.length() < 8) {
input += key;

lcd.setCursor(6 + input.length() - 1, 1);
lcd.print("*");
}
}
}
}
```

6. Explanation of Important Program Sections

6.1 setup() function

```
void setup() {
lcd.begin(16, 2);
lockServo.attach(servoPin);
lockServo.write(LOCK_POSITION);
welcomeScreen();
}
```

setup() runs once after power-up or reset. lcd.begin(16, 2) initializes a 16-column, two-row LCD. lockServo.attach(A3) assigns the servo signal pin. lockServo.write(0) commands the initial locked position. welcomeScreen() displays the startup message and then the password prompt.

6.2 Reading a keypad key

```
char key = keypad.getKey();
```

This asks the keypad library whether a key has been pressed. If a key is available, the returned character is handled by the if/else conditions.

6.3 Submitting the password

```
if (key == '#') {
if (input == password) {
// Correct password
} else {
// Incorrect password
}
}
```

The # key acts like Enter. The program compares the complete input string with the stored password. The characters and their order must match exactly.

6.4 Clearing and masking

```
else if (key == '*') {
input = "";
} else {
input += key;
lcd.print("*");
}
```

The * key clears the input string. For other keys, the character is added to input, but the LCD shows an asterisk to hide the password.

6.5 Servo control

```
lockServo.write(UNLOCK_POSITION);
delay(5000);
lockServo.write(LOCK_POSITION);
```

The servo is commanded to 90°, waits five seconds, and returns to 0°. The delay is blocking: the Arduino does not process keypad input during those five seconds.

7. Building the Circuit in Proteus

11. Create a new Proteus project and open the schematic editor.
12. Press P (Pick Devices) and search for Arduino UNO, a 4×3 keypad, LM016L (or another 16×2 LCD), and a positional servo model available in your library (for example, MOTOR-PWMSER if that is the model you have).
13. Place the components on the schematic.
14. Connect the keypad row and column lines according to Section 3.
15. Connect LCD RS, EN, and D4–D7 according to Section 3. Connect LCD power, ground, RW, contrast, and backlight pins as described in Section 3.1.
16. Connect the servo signal to A3. Connect its supply and ground according to the model pin labels.
17. Check every connection against the tables before running the simulation.

Proteus libraries can contain different servo models with different pin labels. Check the selected model's visible pin names and properties rather than assuming all servo models have the same pin order.

8. Compile the Code and Generate a HEX File

18. Open Arduino IDE and create a new sketch.
19. Paste the complete program from Section 5.
20. Make sure the Keypad and Servo libraries are installed. LiquidCrystal is normally included with the Arduino AVR board package.
21. Select Tools → Board → Arduino AVR Boards → Arduino Uno.
22. Click Verify (the checkmark) to compile the sketch.
23. Choose Sketch → Export Compiled Binary.
24. Choose Sketch → Show Sketch Folder to open the sketch directory.
25. Find the generated .ino.hex file. Use the newly generated file for this version of the project.

The exact HEX filename and build folder depend on your sketch name and Arduino IDE version. Do not reuse a HEX file generated from an older version of the code.

9. Load the HEX File into Proteus

26. Stop the Proteus simulation if it is running.
27. Double-click the Arduino UNO component.
28. Find the Program File field and browse to the newly generated .hex file.
29. Set the clock/frequency to 16 MHz if the property is available.
30. Confirm the properties and click Run.

10. Testing Procedure and Expected Results

Test	Keypad action	LCD result	Servo result
Power-up	No key pressed	Welcome To / Smart Door Lock, then Enter Password: / Pass:	Locked at 0°.
Correct password	Press 1, 2, 3, 4, then #	Access Granted / Door Unlocked; then Door Locked	Moves to 90° for 5 seconds, then returns to 0°.
Incorrect password	Enter any incorrect sequence, then #	Wrong Password! / Access Denied	Remains locked.

Clear input	Enter some digits, then press *	Password entry line is cleared	Remains locked.
Empty submission	Press # without entering digits	Wrong Password! / Access Denied	Remains locked.

10.1 Example successful test

LCD: Enter Password:
Pass:

Keypad: 1 2 3 4 #

LCD: Access Granted
Door Unlocked

After 5 seconds:
LCD: Door Locked

11. Troubleshooting

Problem	Possible reason	What to check
LCD is blank	Power, contrast, or wiring problem	Check VSS, VDD, RW, V0/VEE, RS, EN, and D4–D7. Adjust contrast.
LCD shows blocks only	Contrast or initialization problem	Adjust contrast and verify LCD wiring and constructor pin order.
Keypad does not respond	Row/column wiring or mapping mismatch	Check R1–R4 and C1–C3 against the code and component pin labels.
Wrong keypad character appears	Keypad connector order differs	Verify the keypad pin order and update rowPins/colPins if necessary.
Servo does not move	Signal, supply, ground, or model mismatch	Confirm signal is A3, supply is correct, and grounds are common.
Servo turns the wrong way for locking	Mechanical orientation differs	Adjust LOCK_POSITION and UNLOCK_POSITION for the mechanism.
Proteus runs an old program	Old HEX file remains selected	Export the compiled binary again and select the latest .ino.hex file.

12. Important Notes and Limitations

• This final project version does not include an ultrasonic sensor, distance measurement, or automatic person detection. • The password is fixed in the source code as 1234. To change it, edit the password variable and compile the sketch again. • The 0° and 90° servo positions are example values. Adjust them to fit the physical lock mechanism.

• The five-second delay blocks keypad processing while the servo is in the unlocked interval.

• This is an educational prototype. A real door lock requires a mechanically safe design, a suitable power supply, secure mounting, and appropriate fail-safe provisions.

13. Viva and Lab Discussion Questions

31. What is the role of the Arduino UNO in this project?
32. Why is a matrix keypad used?
33. What is the purpose of the # key?

34. What is the purpose of the * key?
35. Why does the LCD display asterisks instead of the actual password?
36. What does Servo.attach() do?
37. What does Servo.write() do?
38. Why does the program use delay(5000)?
39. What happens when the entered password is incorrect?
40. Which LCD pins are used for 4-bit communication?

14. Project Summary

This laboratory exercise demonstrates digital input using a matrix keypad, character output using an LCD, conditional password checking, and actuator control using a servo motor. Arduino UNO receives the password, compares it with the stored value, and controls the servo according to the result. Proteus can be used to simulate and verify the circuit before physical implementation.